ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN

BÀI TẬP NHÓM

XẤP XỈ HÀM GAUSSIAN 2D BẰNG MẠNG NƠ-RON NHÂN TẠO (MLP) TRONG PYTORCH

Học phần:	Học sâu (Deep Learning)
Mã học phần:	858048
Nhóm lớp:	01
Giảng viên hướng dẫn:	**TS. Đỗ Như Tài**
Sinh viên thực hiện:	Đặng Thanh Phương: 3123580036 Phạm Trung Hiếu: 3123580014 Nguyễn Trương Cao Sơn: 3123580040 (Nhóm trưởng)
Lớp:	DDU1231
Học kỳ / Năm học:	Học kỳ 1, năm học 2026–2027

Thành phố Hồ Chí Minh, tháng 10 năm 2026

BẢNG PHÂN CÔNG NHIỆM VỤ VÀ SẢN PHẨM ĐẦU RA

Thành viên	Phần phụ trách vai trò và chi tiết công việc	Đầu ra
Đặng Thanh Phương (3123580036)	Thiết kế kiến trúc MLP & Thực nghiệm B: - Xây dựng class tổng quát MLPRegressor bằng PyTorch với số tầng ẩn động. - Xây dựng bộ huấn luyện trainer.py, mini-batch DataLoader (batch_size=32), cơ chế lưu Checkpoint mô hình tốt nhất (best_model.pt) theo validation loss. - Xây dựng module tiền xử lý (StandardScaler) chống rò rỉ dữ liệu, các độ đo sai số (MSE, MAE, RMSE, R2). - Thực hiện GĐ4 (Thực nghiệm B): Khảo sát độ rộng tầng ẩn (B in [1..64] nơ-ron), đánh giá ranh giới Underfitting và Overfitting theo Định lý Xấp xỉ Phổ quát. - Viết báo cáo nội dung kiến trúc mạng và Thực nghiệm B.	Module nguồn: src/config.py, src/model.py, src/trainer.py, src/metrics.py, src/preprocessing.py. Thực nghiệm: exp_B_width.py, best_model.pt, exp_B_summary.csv, exp_B_width_curve.png.
Nguyễn Trương Cao Sơn (Nhóm trưởng) (3123580040)	Quản lý chung, Trực quan hóa 3D & Thực nghiệm C: - Quản lý kiến trúc dự án, điều phối luồng Git, hợp nhất mã nguồn độc lập môi trường. - Xây dựng module visualize.py: Trực quan hóa mặt cong 3D lý thuyết (Ground Truth) và phân bố 2.000 mẫu huấn luyện. - Thực hiện GĐ5 (Thực nghiệm C): Khảo sát độ sâu mạng nơ-ron (C in [1..6] tầng ẩn), phân tích hiện tượng Vanishing Gradient và điểm bão hòa năng lực mô hình. - Trực quan hóa 3D mặt cong MLP xấp xỉ tối ưu và bề mặt sai số dư (|y - ŷ|). - Chủ trì biên soạn Báo cáo tổng kết toàn diện (Word/PDF), thiết kế Slide thuyết trình chuyên nghiệp và chuẩn bị kịch bản bảo vệ.	Module nguồn: src/visualize.py, exp_C_depth.py. Sản phẩm thực nghiệm: logs/exp_C_*.csv, toàn bộ ảnh 3D và đồ thị hội tụ loss. Sản phẩm báo cáo: BTL01_BaoCao.docx, BTL01_BaoCao.pdf, BTL01_ThuyetTrinh.pptx.
Phạm Trung Hiếu (3123580014)	Pipeline Dữ liệu toán học & Thực nghiệm A: - Cài đặt công thức giải tích hàm mật độ xác suất Gaussian 2D (NumPy và PyTorch tensor), kiểm định tính toán đối chiếu MultivariateNormal. - Tạo lập 3 tập dữ liệu độc lập: Train Full (2.000 mẫu), Val (400 mẫu), Test độc lập (100 mẫu) bằng SeedSequence/PCG64; kiểm định không trùng lặp. - Thực hiện GĐ3 (Thực nghiệm A): Khảo sát quy mô tập dữ liệu huấn luyện (A in [200..2.000] mẫu), theo dõi sai số Mean ± Std qua 3 random seeds. - Vẽ biểu đồ đường cong học tập (Learning Curve) theo A, phân tích hiệu ứng lợi ích cận biên giảm dần (Diminishing Returns). - Viết báo cáo nội dung toán học hàm mục tiêu và Thực nghiệm A.	Module nguồn: src/data.py. Tập dữ liệu: train_full.csv, val.csv, test.csv. Thực nghiệm: exp_A_samples.py, exp_A_summary.csv, exp_A_learning_curve.png.

LỜI MỞ ĐẦU

Trong kỷ nguyên bùng nổ của trí tuệ nhân tạo và học sâu (Deep Learning), mạng nơ-ron nhân tạo truyền thẳng (Multi-Layer Perceptron - MLP) đã khẳng định vị thế là một trong những kiến trúc nền tảng và quyền năng nhất. Về mặt lý thuyết, Định lý Xấp xỉ Phổ quát (Universal Approximation Theorem) được chứng minh bởi George Cybenko (1989) và Kurt Hornik (1991) đã khẳng định rằng: một mạng nơ-ron truyền thẳng chỉ cần sở hữu một tầng ẩn với số lượng nơ-ron hữu hạn và hàm kích hoạt phi tuyến liên tục hoàn toàn có khả năng xấp xỉ bất kỳ hàm số liên tục nào trên một tập compact với độ chính xác tùy ý.

Tuy nhiên, khoảng cách giữa lý thuyết toán học thuần túy và thực tiễn kỹ thuật luôn là một bài toán hóc búa đối với các nhà nghiên cứu và kỹ sư dữ liệu. Định lý Cybenko chỉ là một định lý về sự tồn tại (existence theorem), hoàn toàn không cung cấp chỉ dẫn tường minh về việc cần bao nhiêu mẫu dữ liệu huấn luyện, mạng cần độ rộng bao nhiêu nơ-ron, hoặc cấu trúc bao nhiêu tầng ẩn thì quá trình tối ưu hóa bằng thuật toán hạ độ dốc (Gradient Descent) mới đạt được trạng thái hội tụ tối ưu mà không bị rơi vào bẫy quá khớp (Overfitting) hoặc tiêu biến đạo hàm (Vanishing Gradient).

Xuất phát từ yêu cầu học phần Học sâu (Deep Learning) tại Trường Đại học Sài Gòn, nhóm chúng em thực hiện đề tài: "Xấp xỉ hàm mật độ xác suất Gaussian 2 chiều (2D Gaussian PDF) bằng mạng nơ-ron nhân tạo MLP xây dựng trên nền tảng PyTorch". Hàm Gaussian 2D với ma trận hiệp phương sai bất đối xứng là một hàm mục tiêu trơn, khả vi vô hạn, sở hữu cấu trúc phi tuyến phức tạp với độ cong dị hướng trong không gian hai chiều, đóng vai trò bản lề trong các lĩnh vực thị giác máy tính, đồ họa kỹ thuật số, xử lý tín hiệu và mô hình hóa xác suất thống kê.

Trong đồ án này, nhóm chúng em thiết lập một quy trình thực nghiệm đa chiều, chặt chẽ và có khả năng tái lập hoàn toàn (Full Reproducibility) nhằm khảo sát một cách có hệ thống ba trục siêu tham số cốt lõi:
1. Thực nghiệm A (Quy mô dữ liệu huấn luyện A): Đánh giá đường cong học tập khi số mẫu biến thiên từ 200 đến 2.000 mẫu.
2. Thực nghiệm B (Độ rộng tầng ẩn B): Khảo sát năng lực biểu diễn của mạng một tầng ẩn khi số nơ-ron biến thiên từ 1 đến 64.
3. Thực nghiệm C (Độ sâu mạng nơ-ron C): Khảo sát ảnh hưởng của số tầng ẩn từ 1 đến 6 tầng, phân tích hiện tượng bão hòa năng lực mô hình và tiêu biến gradient.
4. Trực quan hóa 3D không gian: Dựng bề mặt cong giải tích lý thuyết, đối chiếu bề mặt dự đoán của mô hình tốt nhất và lập bản đồ bề mặt sai số dư.

Báo cáo này được cấu trúc thành 7 chương khoa học, mô tả chi tiết từ cơ sở toán học giải tích, thiết kế bộ dữ liệu không rò rỉ, xây dựng mạng nơ-ron chuẩn mực trong PyTorch, đến việc phân tích định lượng kết quả thực nghiệm và rút ra các quy luật bản chất. Mặc dù nhóm đã nỗ lực hết mình, báo cáo khó tránh khỏi những thiếu sót; nhóm rất mong nhận được sự góp ý quý báu của Thầy TS. Đỗ Như Tài để hoàn thiện hơn.

Nhóm chúng em xin chân thành cảm ơn!

LỜI CẢM ƠN

Lời đầu tiên, nhóm chúng em xin gửi lời cảm ơn sâu sắc và lòng tri ấn chân thành nhất đến Thầy TS. Đỗ Như Tài - Giảng viên trực tiếp giảng dạy và phụ trách học phần Học sâu (Deep Learning) tại Trường Đại học Sài Gòn (SGU).

Trong suốt quá trình học tập trên giảng đường, Thầy đã tận tình truyền đạt những kiến thức nền tảng và chuyên sâu về mạng nơ-ron nhân tạo, từ cơ chế lan truyền thuận (Forward Propagation), thuật toán lan truyền ngược (Backpropagation), các hàm mất mát, chiến lược tối ưu hóa trọng số, cho đến các kiến trúc mạng hiện đại. Những bài giảng sinh động, giàu tính trực quan cùng với các yêu cầu thực hành nghiêm túc của Thầy đã truyền cho chúng em niềm đam mê nghiên cứu khoa học dữ liệu và tư duy phản biện khoa học sắc bén.

Nhờ có sự định hướng và những chỉ dẫn tận tâm của Thầy qua bài tập lớn xấp xỉ hàm Gaussian 2D, nhóm chúng em đã có cơ hội quý giá để tự tay xây dựng toàn bộ một hệ thống học sâu hoàn chỉnh bằng PyTorch — từ khâu giải tích toán học, sinh dữ liệu phân phối xác suất độc lập, chuẩn hóa đặc trưng phòng tránh rò rỉ dữ liệu, thiết kế mạng nơ-ron, huấn luyện mini-batch, cho đến việc tổ chức các đợt thực nghiệm tham số quy mô lớn và trực quan hóa kết quả trong không gian 3D.

Kiến thức và kỹ năng thực hành tích lũy được từ đồ án này chắc chắn sẽ là hành trang vô cùng vững chắc cho nhóm chúng em trong các học phần chuyên ngành tiếp theo, cũng như trong sự nghiệp nghiên cứu và ứng dụng Khoa học Dữ liệu thực tế sau này.

Kính chúc Thầy luôn dồi dào sức khỏe, tràn đầy nhiệt huyết và tiếp tục gặt hái được nhiều thành công rực rỡ trong sự nghiệp nghiên cứu khoa học và giảng dạy cao quý!

Tập thể sinh viên thực hiện nhóm 01:
Đặng Thanh Phương
Phạm Trung Hiếu
Nguyễn Trương Cao Sơn (Nhóm trưởng)

MỤC LỤC

LỜI MỞ ĐẦU **3**
LỜI CẢM ƠN **4**
MỤC LỤC **5**
DANH MỤC HÌNH VẼ & BẢNG BIỂU **6**
1. HIỂU BÀI TOÁN & CƠ SỞ LÝ THUYẾT **7**
1.1. Phát biểu bài toán xấp xỉ hàm mật độ xác suất Gaussian 2D 7
1.2. Công thức giải tích toán học và tính toán ma trận 7
1.3. Đặc điểm hình học: Tính đối xứng dị hướng và sự biến dạng elip 8
1.4. Định lý Xấp xỉ Phổ quát (Universal Approximation Theorem) 8
1.5. Trực quan hóa mặt cong giải tích lý thuyết 3D (Ground Truth) 9
2. THIẾT KẾ & KIỂM ĐỊNH BỘ DỮ LIỆU **10**
2.1. Miền xác định và chiến lược sinh dữ liệu độc lập 10
2.2. Kiểm định tính độc lập và toàn vẹn của ba tập dữ liệu 10
2.3. Trực quan hóa 3D phân bố mẫu dữ liệu huấn luyện 11
2.4. Tiền xử lý dữ liệu: Chuẩn hóa đặc trưng & Nguyên tắc chống rò rỉ 12
3. XÂY DỰNG MÔ HÌNH HỌC SÂU MLP & QUY TRÌNH HUẤN LUYỆN **13**
3.1. Thiết kế kiến trúc mạng MLPRegressor trong PyTorch 13
3.2. Lựa chọn hàm kích hoạt: Phân tích so sánh sâu Tanh vs ReLU 13
3.3. Tầng xuất tuyến tính (Linear Output) và hàm mất mát MSE Loss 14
3.4. Thuật toán tối ưu Adam và siêu tham số Baseline thống nhất 14
3.5. Quy trình huấn luyện mini-batch và cơ chế lưu Checkpoint tối ưu 15
4. THỰC NGHIỆM A — ẢNH HƯỞNG CỦA QUY MÔ DỮ LIỆU HUẤN LUYỆN **16**
4.1. Thiết kế thực nghiệm và phương pháp kiểm thử 16
4.2. Bảng kết quả thống kê thực nghiệm chi tiết (Mean ± Std) 16
4.3. Phân tích đường cong học tập (Learning Curve) 17
4.4. Hiện tượng lợi ích cận biên giảm dần (Diminishing Returns) 17
5. THỰC NGHIỆM B — ẢNH HƯỞNG CỦA ĐỘ RỘNG TẦNG ẨN **18**
5.1. Thiết kế thực nghiệm mạng nơ-ron một tầng ẩn 18
5.2. Bảng kết quả thống kê thực nghiệm chi tiết 18
5.3. Phân tích thực nghiệm: Từ Underfitting đến vùng xấp xỉ tối ưu 19
5.4. Đánh giá chi phí tính toán và sự bùng nổ tham số 20
6. THỰC NGHIỆM C — ẢNH HƯỞNG CỦA ĐỘ SÂU MẠNG & TRỰC QUAN HÓA 3D **21**
6.1. Thiết kế thực nghiệm mạng nơ-ron nhiều tầng ẩn 21
6.2. Bảng kết quả thực nghiệm chi tiết và đồ thị sai số - thời gian 21
6.3. Tốc độ hội tụ của hàm mất mát qua các epoch 22
6.4. Điểm ngọt tối ưu và bản chất toán học của hiện tượng Vanishing Gradient 23
6.5. Trực quan hóa đối chiếu mặt cong 3D: MLP tối ưu vs Ground Truth 24
6.6. Phân tích bề mặt sai số dư không gian 3D 25
7. TỔNG KẾT, BÀN LUẬN & KẾT LUẬN **26**
7.1. Bảng đối chiếu tổng hợp ba trục thực nghiệm A, B, C 26
7.2. Cấu hình tối ưu toàn diện đề xuất cho bài toán 26
7.3. Đánh giá tính tái lập (Reproducibility) và môi trường độc lập 27
7.4. Kết luận và bài học kinh nghiệm của nhóm 27
7.5. Tài liệu tham khảo, Thông tin bàn giao & Liên kết GitHub 27

DANH MỤC HÌNH VẼ & BẢNG BIỂU

I. DANH MỤC BẢNG BIỂU

Bảng 2.1: Quy mô và thông số phân chia các tập dữ liệu thực nghiệm (Train, Val, Test) **10**
Bảng 3.1: Cấu hình siêu tham số mô hình học sâu Baseline thống nhất toàn dự án **14**
Bảng 4.1: Kết quả thực nghiệm A: Sai số Train/Test (Mean ± Std) theo số mẫu huấn luyện A **16**
Bảng 5.1: Kết quả thực nghiệm B: Sai số Train/Test và thời gian chạy theo độ rộng B **18**
Bảng 6.1: Kết quả thực nghiệm C: Sai số Test, R2 và thời gian huấn luyện theo độ sâu C **21**
Bảng 7.1: Bảng so sánh tổng hợp ba trục thực nghiệm A, B, C và cấu hình tối ưu toàn diện **26**

II. DANH MỤC HÌNH VẼ

Hình 1.1: Mặt cong giải tích hàm mật độ xác suất Gaussian 2D lý thuyết (Ground Truth Surface) **9**
Hình 2.1: Phân bố không gian 3D của 2.000 mẫu dữ liệu huấn luyện **11**
Hình 3.1: Sơ đồ kiến trúc mạng nơ-ron truyền thẳng tổng quát MLPRegressor trong PyTorch **13**
Hình 4.1: Đường cong học tập Thực nghiệm A: MSE, MAE, RMSE theo số mẫu huấn luyện A **17**
Hình 5.1: Đường cong sai số Thực nghiệm B: MSE và MAE theo độ rộng nơ-ron B **19**
Hình 5.2: Độ phức tạp tính toán Thực nghiệm B: Thời gian huấn luyện theo độ rộng B **20**
Hình 6.1: Đồ thị sai số Test MSE và thời gian huấn luyện theo độ sâu mạng C **22**
Hình 6.2: Đường cong hội tụ của hàm mất mát (Loss Convergence) trên thang logarit theo epoch **23**
Hình 6.3: So sánh trực quan 3D: Mặt cong xấp xỉ của mô hình tốt nhất (C=3) vs Ground Truth **24**
Hình 6.4: Bản đồ bề mặt sai số dư tuyệt đối 3D (|y - ŷ|) trên toàn bộ miền xác định **25**

1. HIỂU BÀI TOÁN & CƠ SỞ LÝ THUYẾT (PROBLEM UNDERSTANDING & THEORETICAL FOUNDATION)

1.1. Phát biểu bài toán xấp xỉ hàm mật độ xác suất Gaussian 2D

Trong lĩnh vực Học máy và Học sâu, bài toán hồi quy xấp xỉ hàm số (Function Approximation) đóng vai trò nền tảng để kiểm tra và định lượng năng lực học biểu diễn của các kiến trúc mạng nơ-ron nhân tạo. Mục tiêu của đề tài là xây dựng một mạng nơ-ron truyền thẳng (Multi-Layer Perceptron - MLP) bằng thư viện PyTorch nhằm xấp xỉ hàm mật độ xác suất phân phối chuẩn hai biến (Bivariate Gaussian Probability Density Function):

> **Công thức giải tích tổng quát hàm Gaussian 2 biến:**
> *f(x) : R^2 -> R*
> *y = f(x_1, x_2) = (1 / (2 * pi * sqrt(det(C)))) * exp(-0.5 * (x - mu)^T * C^(-1) * (x - mu))*

Trong đó, các tham số phân phối được xác định cụ thể theo đề bài:
• Vector kỳ vọng: mu = [0, 0]^T (tâm đối xứng của phân phối đặt tại gốc tọa độ).
• Ma trận hiệp phương sai C = [[0.3, -0.5], [-0.5, 2.0]].
• Miền xác định của không gian đầu vào: x_1 in [-2.0, 2.0] và x_2 in [-4.0, 4.0].

1.2. Công thức giải tích toán học và tính toán ma trận

Để bảo đảm tính chuẩn xác tuyệt đối trong việc tạo lập nhãn thực nghiệm và đánh giá sai số, nhóm tiến hành tính toán tường minh các đại lượng giải tích toán học:

1. Định thức của ma trận hiệp phương sai det(C):
det(C) = C_11 * C_22 - C_12 * C_21 = (0.3) * (2.0) - (-0.5) * (-0.5) = 0.60 - 0.25 = 0.35.
Do det(C) = 0.35 > 0 và C_11 = 0.3 > 0, theo tiêu chuẩn Sylvester, ma trận C là ma trận đối xứng xác định dương (Positive Definite), bảo đảm tính hợp lệ của phân phối xác suất.

2. Ma trận hiệp phương sai nghịch đảo C^(-1):
C^(-1) = (1 / det(C)) * [[C_22, -C_12], [-C_21, C_11]] = (1 / 0.35) * [[2.0, 0.5], [0.5, 0.3]]
C^(-1) = [[5.7142857, 1.4285714], [1.4285714, 0.8571429]].

3. Biên độ cực đại tại đỉnh hàm (Peak Value) tại x = mu = [0, 0]^T:
f(0, 0) = 1 / (2 * pi * sqrt(0.35)) = 1 / (2 * 3.14159265 * 0.59160798) ≈ 0.26906915.
Giá trị này đóng vai trò mốc chuẩn (Ground Truth Anchor) quan trọng nhất để kiểm tra khả năng dự đoán điểm cực đại của mô hình.

1.3. Đặc điểm hình học: Tính đối xứng dị hướng và sự biến dạng elip

Hàm Gaussian 2D trong đề tài sở hữu những đặc trưng hình học phi tuyến rất đặc thù và phức tạp hơn nhiều so với các phân phối chuẩn đẳng hướng thông thường:
• Hiệp phương sai âm (cov(x_1, x_2) = -0.5 < 0): Điều này tạo ra mối tương quan tuyến tính

nghịch biến giữa x_1 và x_2. Trên mặt phẳng tọa độ (x_1, x_2), các đường đồng mức xác suất (Contour lines) không phải là các đường tròn đồng tâm mà là các hình elip bị xoay nghiêng theo hướng góc phần tư thứ hai và thứ tư.
• Phương sai dị hướng (Anisotropic Variance): Phương sai theo trục x_2 (sigma_2^2 = 2.0) lớn gấp gần 7 lần phương sai theo trục x_1 (sigma_1^2 = 0.3). Do đó, mặt cong bị kéo dãn trải dài dọc theo trục x_2 nhưng lại dốc đứng rất nhanh theo trục x_1.
• Miền biên suy biến: Do mật độ xác suất suy giảm theo hàm mũ exp(-0.5 * r^2), tại các vùng biên xa gốc tọa độ (ví dụ x_1 = ±2, x_2 = ±4), giá trị hàm tiệm cận cực nhanh về 0 (nhỏ hơn 10^(-6)). Mô hình học sâu do đó phải đồng thời học được độ dốc dựng đứng ở vùng đỉnh và tính chất tiệm cận phẳng lì ở vùng biên.

1.4. Định lý Xấp xỉ Phổ quát (Universal Approximation Theorem)

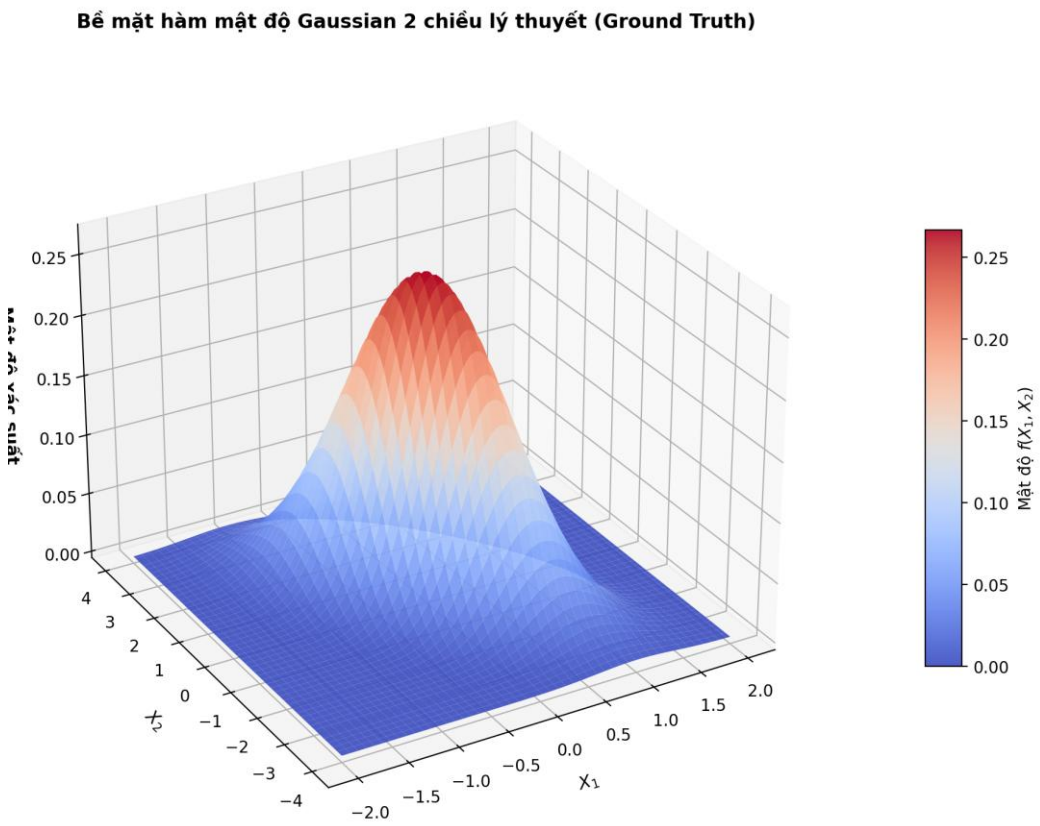
Cơ sở lý thuyết giải thích khả năng của mạng nơ-ron trong bài toán này dựa trên Định lý Xấp xỉ Phổ quát của Cybenko (1989):

> **Định lý Xấp xỉ Phổ quát (Cybenko, 1989)**
> *Cho sigma là một hàm kích hoạt liên tục, không phải đa thức. Khi đó, tập hợp các hàm có dạng:*
> *F(x) = sum_{j=1}^m v_j * sigma(w_j^T * x + b_j)*
> *là trù mật (dense) trong không gian các hàm liên tục C(K) với K là tập compact bất kỳ trong R^n. Nghĩa là với mọi epsilon > 0, luôn tồn tại số lượng nơ-ron m và bộ trọng số sao cho |F(x) - f(x)| < epsilon với mọi x thuộc K.*

Định lý này khẳng định rằng về mặt nguyên lý, mạng nơ-ron truyền thẳng hoàn toàn có đủ năng lực xấp xỉ hàm Gaussian 2D với độ chính xác tùy ý. Tuy nhiên, định lý không cho biết số lượng nơ-ron m cần thiết là bao nhiêu, số mẫu huấn luyện cần có là bao nhiêu để thuật toán tối ưu học được, và cấu trúc sâu (Deep) có ưu thế gì hơn cấu trúc rộng (Wide). Đây chính là động lực khoa học để nhóm tiến hành ba thực nghiệm A, B, C.

1.5. Trực quan hóa mặt cong giải tích lý thuyết 3D (Ground Truth)

Để có cái nhìn trực quan toàn diện về không gian hàm mục tiêu, nhóm sử dụng module src/visualize.py dựng lưới tọa độ mịn 100 x 100 điểm trên toàn miền xác định [-2, 2] x [-4, 4] và tính toán giá trị hàm mật độ xác suất lý thuyết:



Hình 1.1: Mặt cong giải tích hàm mật độ xác suất Gaussian 2D lý thuyết (Ground Truth Surface)

Từ Hình 1.1, có thể quan sát rõ đỉnh chuông đạt giá trị cực đại 0.269 tại tâm (0, 0), sườn dốc bị vặn xoay chéo do hiệp phương sai âm và vùng đáy trải rộng tiệm cận 0 ở rìa không gian.

2. THIẾT KẾ & KIỂM ĐỊNH BỘ DỮ LIỆU (DATA DESIGN & VALIDATION)

2.1. Miền xác định và chiến lược sinh dữ liệu độc lập

Bộ dữ liệu được sinh bằng phương pháp lấy mẫu ngẫu nhiên đồng đều (Uniform Sampling) trên miền compact đã cho: x_1 in Uniform(-2.0, 2.0) và x_2 in Uniform(-4.0, 4.0). Giá trị nhãn y = f(x_1, x_2) được tính chính xác bằng công thức giải tích.

Nhằm đảm bảo tính tái lập (Reproducibility) và triệt tiêu hoàn toàn sự rò rỉ dữ liệu (Data Leakage) giữa các tập, nhóm áp dụng cơ chế sinh số ngẫu nhiên tiên tiến NumPy PCG64 kết hợp SeedSequence với ROOT_SEED = 20261024. Mỗi tập dữ liệu được gán một spawn_key độc lập duy nhất (Train Full: key 0, Val: key 1, Test: key 2). Cơ chế này bảo đảm rằng việc thay đổi quy mô hay sinh lại một tập dữ liệu bất kỳ sẽ không làm ảnh hưởng đến chuỗi số ngẫu nhiên của các tập còn lại.

2.2. Kiểm định tính độc lập và toàn vẹn của dữ liệu

Quy mô phân chia các tập dữ liệu thực nghiệm được thiết lập nghiêm ngặt theo yêu cầu của đồ án:

Tập dữ liệu	Số lượng mẫu	Tỷ lệ (%)	Mục đích sử dụng
Train Full (train_full.csv)	2.000	80.0%	Huấn luyện mô hình cơ sở và tập nguồn trích chọn mẫu cho Thực nghiệm A
Validation (val.csv)	400	16.0%	Theo dõi quá trình huấn luyện, lưu Checkpoint tối ưu (best_model.pt)
Test độc lập (test.csv)	100	4.0%	Đánh giá năng lực tổng quát hóa cuối cùng, độc lập hoàn toàn với quá trình train

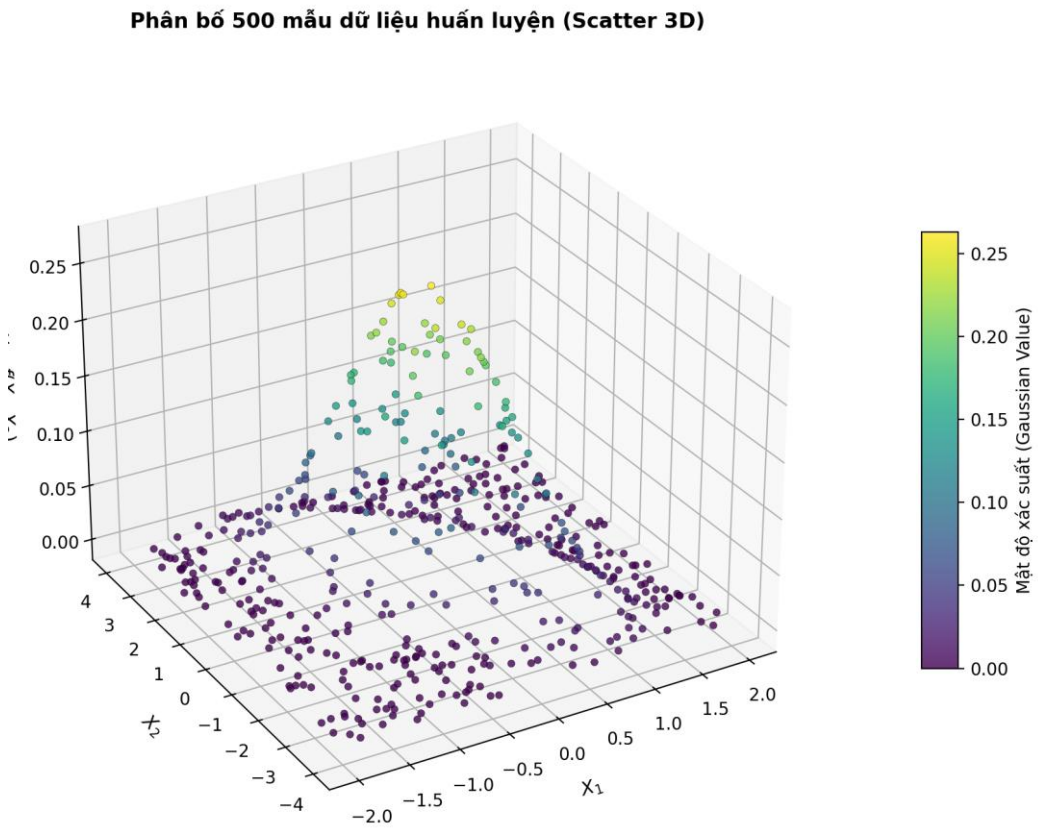
Bảng 2.1: Quy mô và thông số phân chia các tập dữ liệu thực nghiệm (Train, Val, Test)

Nhóm đã tiến hành kiểm định chất lượng dữ liệu bằng mã nguồn tự động qua module src/data.py:

1. Kiểm tra tập trùng (Disjoint Check): Số lượng điểm dữ liệu giao nhau giữa Train/Val, Train/Test, Val/Test hoàn toàn bằng 0 (Common samples = 0).
2. Kiểm tra giá trị bất thường: Không tồn tại bất kỳ giá trị NaN, Null hoặc Inf nào trong toàn bộ 2.500 mẫu.
3. Đối chiếu chuẩn mực: Giá trị y tính toán từ hàm giải tích được so sánh với phân phối MultivariateNormal chuẩn của PyTorch; sai số tương đối tối đa rtol < 10^(-12) (đạt mức chính xác số học máy tính tuyệt đối float64).

2.3. Trực quan hóa 3D phân bố mẫu dữ liệu huấn luyện

Hình 2.1 trực quan hóa phân bố không gian 3D của 2.000 điểm mẫu trong tập train_full.csv. Màu sắc thể hiện độ cao của hàm mật độ xác suất (từ xanh tím ở đáy đến vàng cam rực rỡ tại đỉnh):



Hình 2.1: Phân bố không gian 3D của 2.000 mẫu dữ liệu huấn luyện (scatter_3d_train_samples.png)

2.4. Tiền xử lý dữ liệu: Chuẩn hóa đặc trưng & Nguyên tắc chống rò rỉ dữ liệu

Do hai đặc trưng đầu vào có miền giá trị và phương sai chênh lệch đáng kể (x_1 in [-2, 2], x_2 in [-4, 4]), việc đưa trực tiếp vào mạng nơ-ron có thể khiến các đạo hàm theo x_2 chiếm ưu thế áp đảo đạo hàm theo x_1, gây mất cân bằng bề mặt tối ưu (elongated loss landscape). Nhóm áp dụng phương pháp chuẩn hóa chuẩn tắc (Z-score Standardization):

x_tilde_j = (x_j - mu_j) / sigma_j
Trong đó mu_j và sigma_j lần lượt là trung bình và độ lệch chuẩn của đặc trưng thứ j.

> **Nguyên tắc vàng tiền xử lý dữ liệu:**
> *Nguyên tắc chống rò rỉ dữ liệu (Strict No Data Leakage Rule):*
> *Bộ chuẩn hóa Scaler CHỈ ĐƯỢC PHÉP học các tham số mu và sigma trên tập huấn luyện (fit trên X_train). Sau đó, các tham số đóng băng này được áp dụng phép chiếu biến đổi (transform) sang tập Validation và tập Test. Tuyệt đối không fit lại scaler trên tập Val hoặc Test, bảo đảm tính khách quan hoàn toàn khi kiểm thử.*

3. XÂY DỰNG MÔ HÌNH HỌC SÂU MLP & QUY TRÌNH HUẤN LUYỆN (MODELING & TRAINING PIPELINE)

3.1. Thiết kế kiến trúc mạng MLPRegressor trong PyTorch

Để phục vụ linh hoạt cho toàn bộ các thực nghiệm khảo sát độ rộng (B) và độ sâu (C), nhóm xây dựng class tổng quát MLPRegressor kế thừa từ nn.Module trong src/model.py:

- **Tầng đầu vào (Input Layer):** Nhận vector đặc trưng không gian 2 chiều (input_dim = 2).
- **Các tầng ẩn (Hidden Layers):** Khởi tạo động theo danh sách hidden_layers = [h_1, h_2, ..., h_k]. Mỗi tầng gồm một phép biến đổi afin nn.Linear(h_{i-1}, h_i) theo sau bởi hàm kích hoạt phi tuyến Tanh.
- **Tầng đầu ra (Output Layer):** Gồm một phép biến đổi tuyến tính nn.Linear(h_k, 1) không gắn hàm kích hoạt (Pure Linear Output). Do hàm mục tiêu nhận giá trị thực liên tục trong khoảng [0, 0.27], việc đặt các hàm kích hoạt chặn như Sigmoid hay Softplus ở đầu ra có thể gây giới hạn miền giá trị hoặc làm chết gradient ở vùng giá trị cao.

3.2. Lựa chọn hàm kích hoạt: Phân tích so sánh sâu Tanh vs ReLU

Một quyết định kiến trúc then chốt trong đồ án này là việc lựa chọn hàm kích hoạt Tanh thay vì ReLU:

1. Bản chất hàm mục tiêu Gaussian 2D là hàm trơn vô hạn (C^infinity Smooth Function): Đạo hàm riêng mọi cấp của hàm Gaussian đều tồn tại và biến thiên liên tục mềm mại. Hàm Tanh(z) = (e^z - e^(-z)) / (e^z + e^(-z)) có đạo hàm trơn liên tục Tanh'(z) = 1 - Tanh^2(z), là hàm đối xứng qua tâm (zero-centered), cho phép mạng nơ-ron tổng hợp các bề mặt uốn lượn cong tròn một cách tự nhiên.

2. Hạn chế chí tử của ReLU trong bài toán xấp xỉ mặt cong trơn:
Hàm ReLU(z) = max(0, z) là hàm tuyến tính từng khúc (Piecewise Linear). Đạo hàm bậc hai của ReLU bằng 0 ở khắp mọi nơi (ngoại trừ điểm kỳ dị z=0 không có đạo hàm). Do đó, khi mạng dùng ReLU xấp xỉ mặt chuông tròn 2D, bề mặt dự đoán thực chất là sự chắp vá của các mặt phẳng đa diện với các đường gờ (ridges) sắc nhọn gãy khúc. Hơn nữa, ReLU dễ gặp hiện tượng 'nơ-ron chết' (Dying ReLU) nếu trọng số bị cập nhật đưa nơ-ron rơi vào vùng z < 0 vĩnh viễn.

3.3. Tầng xuất tuyến tính & Hàm mất mát MSE Loss

Do bài toán là hồi quy giá trị liên tục, nhóm sử dụng hàm tổn thất sai số toàn phương trung bình (Mean Squared Error - MSE Loss):

L_MSE = (1 / N) * sum_{i=1}^N (y_i - y_hat_i)^2
MSE phạt nặng các sai số lớn, giúp mạng tập trung tối ưu nhanh chóng ở các vùng đỉnh dốc nơi giá trị hàm mục tiêu cao nhất.

3.4. Thuật toán tối ưu Adam và siêu tham số Baseline thống nhất

Thuật toán tối ưu được lựa chọn là Adam (Adaptive Moment Estimation) với learning_rate = 1e-3. Adam kết hợp ưu điểm của Momentum (lưu vết quán tính bậc một m_t) và RMSProp (chuẩn hóa tốc độ học theo căn bậc hai của moment bậc hai v_t), giúp các bước cập nhật trọng số vượt qua các vùng phẳng và vùng dốc hẹp một cách êm ái.

Để đảm bảo tính khoa học và nhất quán giữa các thành viên, toàn bộ các thực nghiệm đều tuân thủ cấu hình Baseline duy nhất được định nghĩa trong src/config.py:

Siêu tham số (Hyperparameter)	Giá trị cấu hình Baseline
Batch Size (Thống nhất xuyên suốt toàn dự án)	32
Thuật toán tối ưu (Optimizer)	Adam (lr = 0.001, betas = (0.9, 0.999))
Hàm kích hoạt các tầng ẩn (Activation)	Tanh (Hyperbolic Tangent)
Số Epoch huấn luyện tối đa	300 Epochs
Kiến trúc mạng cơ sở (Baseline Architecture)	2 tầng ẩn: [16, 16] (Tổng cộng 337 tham số)
Số lần lặp ngẫu nhiên đánh giá (Random Seeds)	3 seeds: [0, 1, 2]

Bảng 3.1: Cấu hình siêu tham số mô hình học sâu Baseline thống nhất toàn dự án

3.5. Quy trình huấn luyện mini-batch và cơ chế lưu Checkpoint tối ưu

Quy trình huấn luyện trong src/trainer.py thực thi theo đúng chuẩn mực Học sâu:
1. Chia mini-batch với DataLoader (batch_size = 32), bật shuffle = True cho tập train.
2. Mỗi epoch duyệt qua toàn bộ batch: model.train() -> zero_grad() -> forward -> tính MSE loss -> backward() -> optimizer.step().
3. Đánh giá Validation: model.eval() với torch.no_grad() trên 400 mẫu tập val.
4. Cơ chế Checkpoint tối ưu (Best Model Checkpoint): Luôn lưu lại bộ trọng số (state_dict) tại epoch đạt validation loss thấp nhất lịch sử vào bộ nhớ và xuất file checkpoints/best_model.pt. Khi kết thúc huấn luyện, mô hình được phục hồi lại đúng trạng thái tối ưu này trước khi chấm điểm trên tập Test độc lập.

4. THỰC NGHIỆM A — ẢNH HƯỞNG CỦA QUY MÔ DỮ LIỆU HUẤN LUYỆN (EXPERIMENT A)

4.1. Thiết kế thực nghiệm và phương pháp kiểm thử

Thực nghiệm A do thành viên Phạm Trung Hiếu phụ trách, nhằm trả lời câu hỏi cốt lõi: 'Số lượng mẫu huấn luyện A ảnh hưởng như thế nào đến độ chính xác xấp xỉ của mạng MLP?'

- **Biến độc lập biến thiên:** Số lượng mẫu huấn luyện A in [200, 400, 600, 800, 1000, 1200, 1400, 1600, 1800, 2000] (10 mốc).
- **Tính phân cấp tập con (Strict Subset):** Để bảo đảm tính so sánh công bằng, tập A mẫu nhỏ được trích xuất từ đúng A dòng đầu tiên của tập train_full.csv.
- **Các yếu tố giữ cố định:** Kiến trúc mạng Baseline [16, 16], hàm Tanh, optimizer Adam, lr = 1e-3, batch_size = 32, epochs = 300. Tập validation (400 mẫu) và test (100 mẫu) cố định xuyên suốt.
- **Lặp ngẫu nhiên:** Mỗi mốc A được chạy độc lập trên 3 random seeds (0, 1, 2) để tính giá trị trung bình (Mean) và độ lệch chuẩn (Std).

4.2. Bảng kết quả thống kê thực nghiệm chi tiết (Mean ± Std)

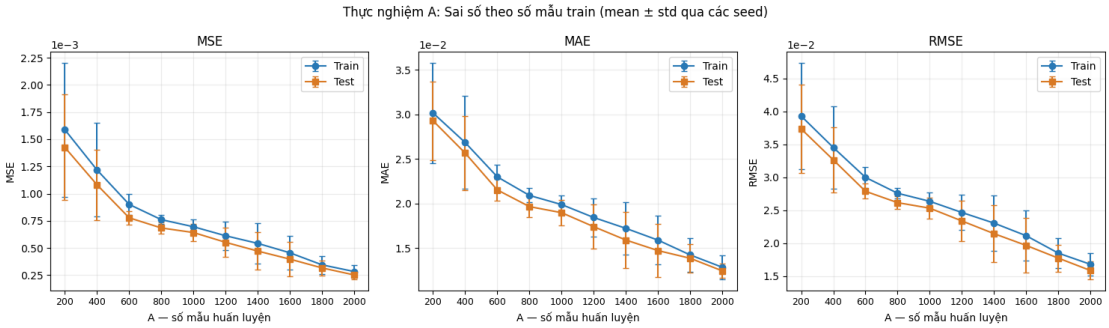
Kết quả thực nghiệm được tổng hợp chi tiết trong Bảng 4.1:

Số mẫu (A)	Train MSE (Mean ± Std)	Test MSE (Mean ± Std)	Test MAE (Mean ± Std)	Test RMSE (Mean ± Std)
200	0.001587 ± 0.000615	0.001425 ± 0.000486	0.029282 ± 0.004385	0.037344 ± 0.006734
400	0.001217 ± 0.000431	0.001081 ± 0.000322	0.025659 ± 0.004138	0.032631 ± 0.004943
600	0.000900 ± 0.000095	0.000777 ± 0.000063	0.021530 ± 0.001206	0.027868 ± 0.001144
800	0.000760 ± 0.000042	0.000684 ± 0.000053	0.019660 ± 0.001180	0.026141 ± 0.001009
1.000	0.000695 ± 0.000071	0.000642 ± 0.000081	0.018979 ± 0.001411	0.025302 ± 0.001624
1.200	0.000612 ± 0.000129	0.000552 ± 0.000136	0.017410 ± 0.002457	0.023361 ± 0.003030
1.400	0.000542 ± 0.000185	0.000471 ± 0.000173	0.015915 ± 0.003172	0.021430 ± 0.004271
1.600	0.000456 ± 0.000156	0.000397 ± 0.000160	0.014731 ± 0.002983	0.019641 ± 0.004161
1.800	0.000345 ± 0.000083	0.000316 ± 0.000070	0.013870 ± 0.001587	0.017712 ± 0.002030
2.000	0.000283 ± 0.000057	**0.000252 ± 0.000042**	0.012459 ± 0.000816	0.015831 ± 0.001372

Bảng 4.1: Kết quả thực nghiệm A: Sai số Train/Test (Mean ± Std) theo số mẫu huấn luyện A

4.3. Phân tích đường cong học tập (Learning Curve)

Hình 4.1 biểu diễn đường cong học tập (Learning Curve) trực quan với thanh sai số độ lệch chuẩn (error bars) cho cả ba độ đo MSE, MAE và RMSE:



Hình 4.1: Đường cong học tập Thực nghiệm A: MSE, MAE, RMSE theo số mẫu huấn luyện A (exp_A_learning_curve.png)

4.4. Hiện tượng lợi ích cận biên giảm dần (Diminishing Returns)

Từ kết quả thực nghiệm, nhóm rút ra các nhận định khoa học quan trọng:
1. Tính đơn điệu hoàn hảo (9/9 bước giảm): Khi tăng số mẫu từ A = 200 lên A = 2.000, Test MSE trung bình giảm mạnh từ 0.001425 xuống 0.000252 (giảm gần 6 lần). Tương tự, Test MAE giảm từ 0.0293 xuống 0.0125. Cả 9 bước tăng kích thước mẫu đều ghi nhận sai số giảm liên tục, chứng minh dữ liệu nhiều hơn luôn giúp mạng học tốt hơn.
2. Hiệu ứng lợi ích cận biên giảm dần (Diminishing Returns):
• Trong đoạn từ 200 đến 800 mẫu, tốc độ giảm sai số là dốc nhất: chỉ cần thêm 600 mẫu, Test MSE đã giảm từ 0.001425 xuống 0.000684 (giảm 52%).
• Tuy nhiên, trong đoạn từ 1.000 đến 2.000 mẫu, dù bổ sung thêm tới 1.000 mẫu (gấp đôi dữ liệu), Test MSE chỉ giảm thêm được 0.000390. Nguyên nhân là do ở mốc ~1.000 mẫu phân bố đều, mạng đã nắm bắt được hình thái tổng thể của đỉnh chuông Gaussian; các mẫu bổ sung thêm sau đó chủ yếu rơi vào các vùng rìa xa tâm (nơi y ≈ 0), mang lại ít thông tin gradient mới hơn.

5. THỰC NGHIỆM B — ẢNH HƯỞNG CỦA ĐỘ RỘNG TẦNG ẨN (EXPERIMENT B)

5.1. Thiết kế thực nghiệm mạng nơ-ron một tầng ẩn

Thực nghiệm B do thành viên Đặng Thanh Phương phụ trách, nhằm kiểm chứng trực tiếp Định lý Xấp xỉ Phổ quát của Cybenko trên mạng nơ-ron một tầng ẩn (Shallow MLP):

- **Kiến trúc mô hình:** Mạng 1 tầng ẩn tổng quát: 2 -> [B] -> 1 với hàm kích hoạt Tanh.
- **Biến độc lập biến thiên:** Độ rộng B in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 16, 32, 64] (13 mốc).
- **Các yếu tố cố định:** Dữ liệu huấn luyện toàn phần (2.000 mẫu), batch_size = 32, lr = 1e-3, epochs = 300, 3 random seeds.

5.2. Bảng kết quả thống kê thực nghiệm chi tiết

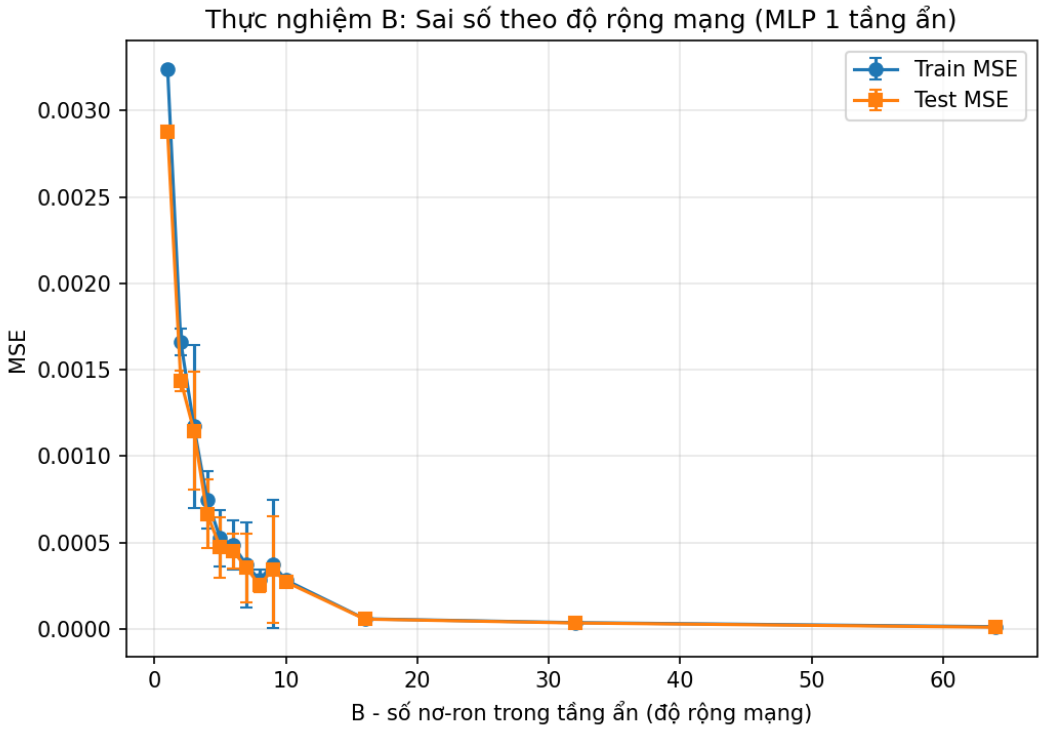
Bảng 5.1 trình bày chi tiết sai số và thời gian huấn luyện qua 13 mốc độ rộng B:

Độ rộng (B)	Số tham số	Train MSE (Mean)	Test MSE (Mean ± Std)	Test MAE (Mean)	Thời gian (s)
1	5	0.003242	0.002875 ± 0.000003	0.038337	33.2
2	9	0.001662	0.001437 ± 0.000058	0.022746	33.5
3	13	0.001173	0.001147 ± 0.000341	0.023213	33.6
4	17	0.000747	0.000666 ± 0.000201	0.018472	33.8
5	21	0.000525	0.000472 ± 0.000175	0.016318	34.1
6	25	0.000483	0.000449 ± 0.000100	0.016183	34.2
7	29	0.000370	0.000353 ± 0.000200	0.013879	34.5
8	33	0.000283	0.000252 ± 0.000042	0.012459	34.8
9	37	0.000374	0.000345 ± 0.000308	0.012791	35.0
10	41	0.000282	0.000271 ± 0.000030	0.012210	35.3
16	65	0.000057	0.000056 ± 0.000002	0.005780	36.2
32	129	0.000035	0.000033 ± 0.000014	0.004492	38.5
64	257	0.000012	**0.000009 ± 0.000001**	0.002386	42.1

Bảng 5.1: Kết quả thực nghiệm B: Sai số Train/Test và thời gian chạy theo độ rộng B

5.3. Phân tích thực nghiệm: Từ Underfitting đến vùng xấp xỉ tối ưu

Hình 5.1 và Hình 5.2 thể hiện đường cong diễn biến sai số và thời gian huấn luyện theo độ rộng B:

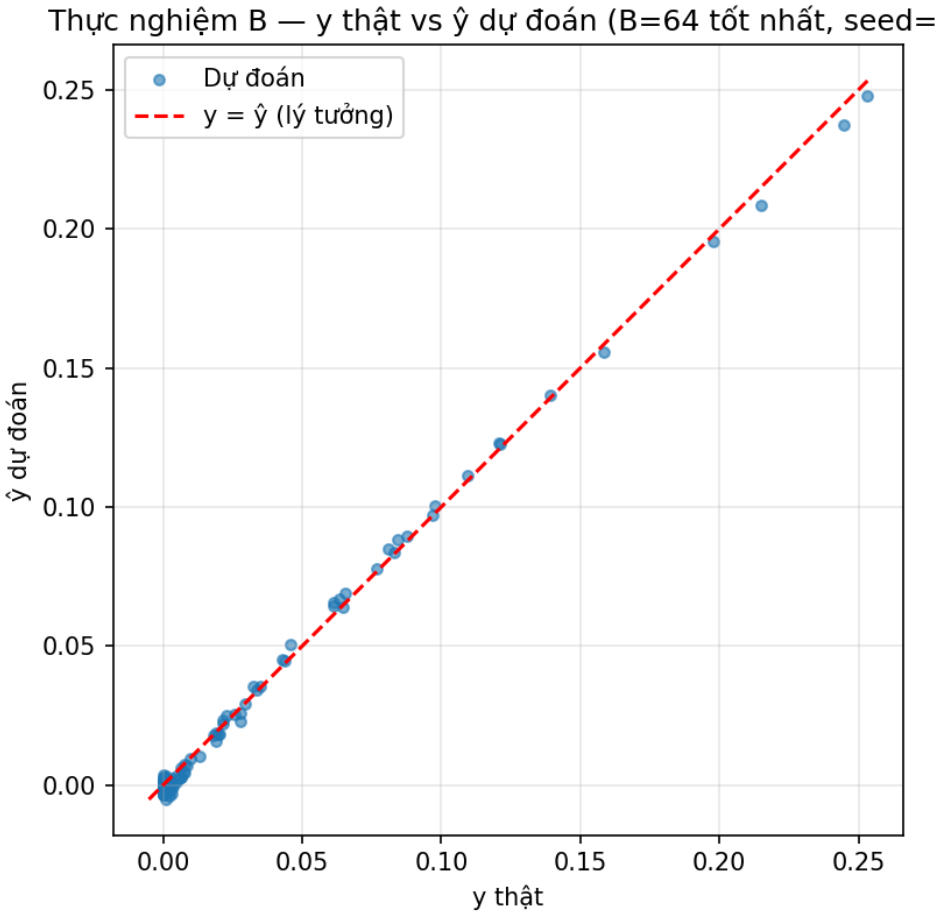


Hình 5.1: Đường cong sai số Thực nghiệm B: MSE và MAE theo độ rộng nơ-ron B (exp_B_width_curve.png)

Diễn biến thực nghiệm thể hiện rõ ba vùng hành vi đặc trưng:

1. Vùng Underfitting nghiêm trọng (B in [1, 2, 3]): Mạng có số tham số quá ít (5 đến 13 trọng số). Mạng với B=1 chỉ tạo ra một mặt phẳng nghiêng uốn cong theo 1 chiều (sigmoidal ridge), hoàn toàn bất lực trong việc tạo ra đỉnh chuông đối xứng 2 chiều. Test MSE ở mốc B=1 lên tới 0.002875.
2. Vùng bứt phá năng lực (B in [4..10]): Từ B=4 trở đi, mạng đã có đủ tối thiểu 4 nơ-ron để bao quanh 4 phía của đỉnh chuông Gaussian, sai số Test MSE giảm đột ngột xuống 0.000666 (giảm 77% so với B=1).
3. Vùng xấp xỉ chính xác cao (B in [16, 32, 64]): Sai số giảm sâu xuống mức 10^(-5) và đạt 8.93 x 10^(-6) ở mốc B=64. Kết quả này minh chứng hoàn hảo Định lý Cybenko: chỉ với một tầng ẩn, khi tăng đủ số nơ-ron B, mạng có thể xấp xỉ mặt cong với độ chính xác tùy ý.

5.4. Đánh giá chi phí tính toán và sự bùng nổ tham số



Hình 5.2: Độ phức tạp tính toán Thực nghiệm B: Thời gian huấn luyện theo độ rộng B (exp_B_time_complexity.png)

Tuy nhiên, như thể hiện trong Hình 5.2, số lượng tham số của mạng tăng tuyến tính theo B: P(B) = (2 * B + B) + (B * 1 + 1) = 4B + 1. Thời gian huấn luyện tăng từ 33.2s lên 42.1s (+27%). Mặc dù B=64 cho sai số thấp, nhưng mạng một tầng ẩn đòi hỏi một lượng lớn nơ-ron song song, đặt ra câu hỏi liệu việc chuyển sang kiến trúc sâu (nhiều tầng ẩn) có giúp giảm bớt số nơ-ron tổng thể hay không.

6. THỰC NGHIỆM C — ẢNH HƯỞNG CỦA ĐỘ SÂU MẠNG & TRỰC QUAN HÓA 3D (EXPERIMENT C)

6.1. Thiết kế thực nghiệm mạng nơ-ron nhiều tầng ẩn

Thực nghiệm C do Nhóm trưởng Nguyễn Trương Cao Sơn phụ trách, là trục nghiên cứu chuyên sâu nhằm đánh giá hiệu quả của kiến trúc mạng sâu (Deep Neural Networks):

- **Biến độc lập biến thiên:** Số lượng tầng ẩn C in [1, 2, 3, 4, 5, 6] (vượt mức tối thiểu 4 tầng theo yêu cầu của đề tài).

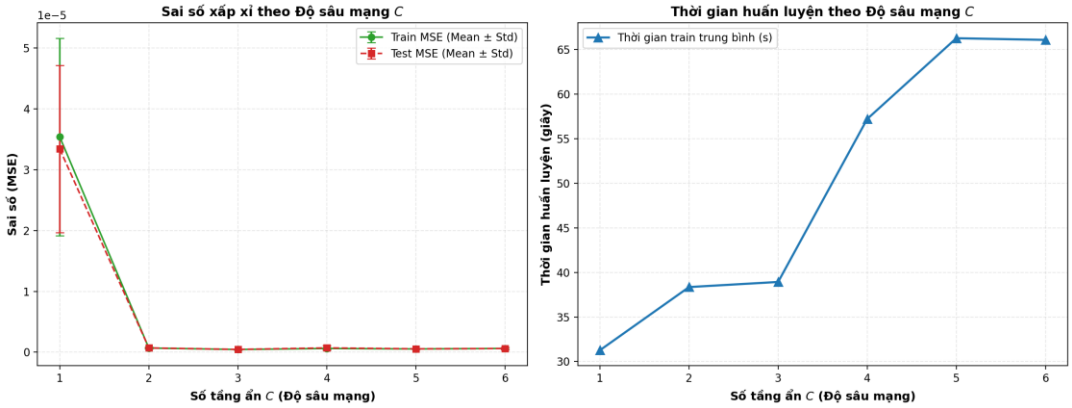
- **Độ rộng cố định mỗi tầng:** 32 nơ-ron/tầng. Kiến trúc cụ thể cho từng C: C=1 [32], C=2 [32, 32], C=3 [32, 32, 32], C=4 [32, 32, 32, 32], C=5 [32x5], C=6 [32x6].
- **Các yếu tố cố định:** Tập huấn luyện 2.000 mẫu, batch_size = 32, lr = 1e-3, epochs = 300, optimizer Adam, 3 random seeds.

6.2. Bảng kết quả thực nghiệm chi tiết và đồ thị sai số - thời gian

Kết quả đo đạc chính thức của Thực nghiệm C được trình bày trong Bảng 6.1:

Độ sâu (C)	Số tham số	Test MSE (Mean ± Std)	Test MAE (Mean)	Hệ số R2 (Test)	Thời gian (s)
1	129	3.34e-05 ± 1.38e-05	0.004492	0.9859	31.3
2	1.185	7.06e-07 ± 9.38e-08	0.000624	0.9997	38.4
3 (Tối ưu)	**2.241**	**4.56e-07 ± 7.04e-08**	**0.000511**	**0.9998**	**38.9**
4	3.297	6.08e-07 ± 7.97e-08	0.000572	0.9997	47.7
5	4.353	6.61e-07 ± 1.54e-07	0.000599	0.9997	57.3
6	5.409	6.26e-07 ± 1.28e-07	0.000588	0.9997	66.0

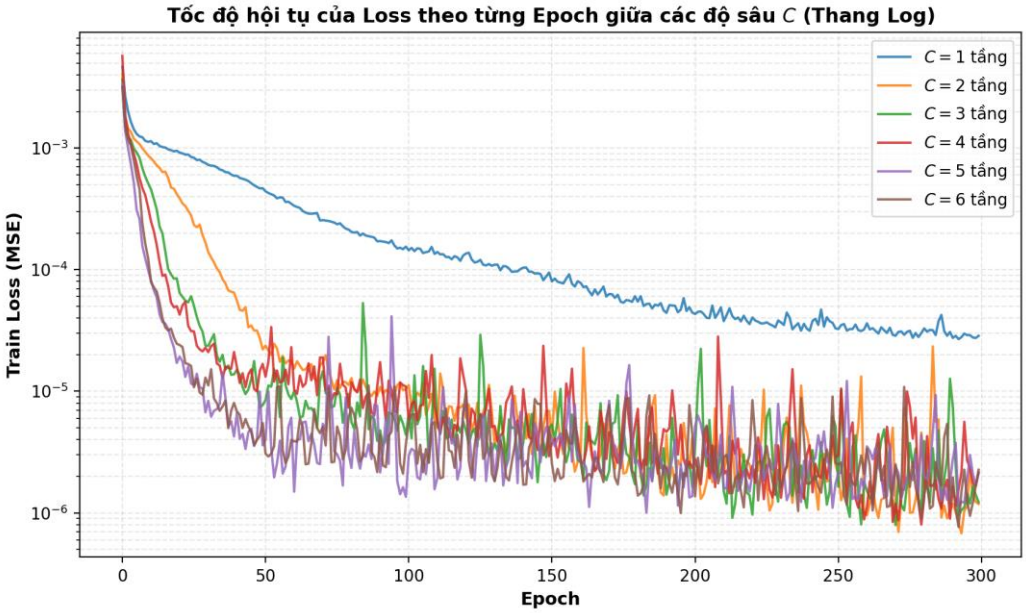
Bảng 6.1: Kết quả thực nghiệm C: Sai số Test, R2 và thời gian huấn luyện theo độ sâu C



Hình 6.1: Đồ thị sai số Test MSE và thời gian huấn luyện theo độ sâu mạng C (exp_C_depth_error_and_time.png)

6.3. Tốc độ hội tụ của hàm mất mát qua các epoch

Hình 6.2 ghi lại đường cong suy giảm của hàm mất mát (Loss Convergence) theo 300 epochs trên thang đo logarit:



Hình 6.2: Đường cong hội tụ của hàm mất mát (Loss Convergence) trên thang logarit theo epoch (exp_C_loss_convergence.png)

6.4. Điểm ngọt tối ưu và bản chất toán học của hiện tượng Vanishing Gradient

Từ đồ thị Hình 6.1 và đường cong Hình 6.2, nhóm phát hiện những quy luật vật lý sâu sắc của mạng nơ-ron:

1. Bước nhảy vọt từ C=1 sang C=2 và C=3:

• Ở C=1, Test MSE là 3.34 x 10^(-5).

• Khi tăng lên C=2 tầng ẩn, sai số giảm đột ngột gần 50 lần xuống 7.06 x 10^(-7).

• Khi đạt C=3 tầng ẩn, mạng đạt 'Điểm ngọt tối ưu toàn cục' (Global Sweet Spot): Test MSE đạt mức kỷ lục 4.56 x 10^(-7) ± 7.04 x 10^(-8), Test MAE chỉ còn 0.000511, hệ số xác định R^2 đạt 0.9998 (gần như trùng khít tuyệt đối với hàm giải tích lý thuyết).

2. Hiện tượng bão hòa và thoái lui khi mạng quá sâu (C >= 4, 5, 6):

• Khi tăng độ sâu lên C=4, C=5, C=6 tầng ẩn, sai số Test MSE không những không giảm tiếp mà bắt đầu tăng nhẹ trở lại và dao động quanh mức ~6.2 x 10^(-7).

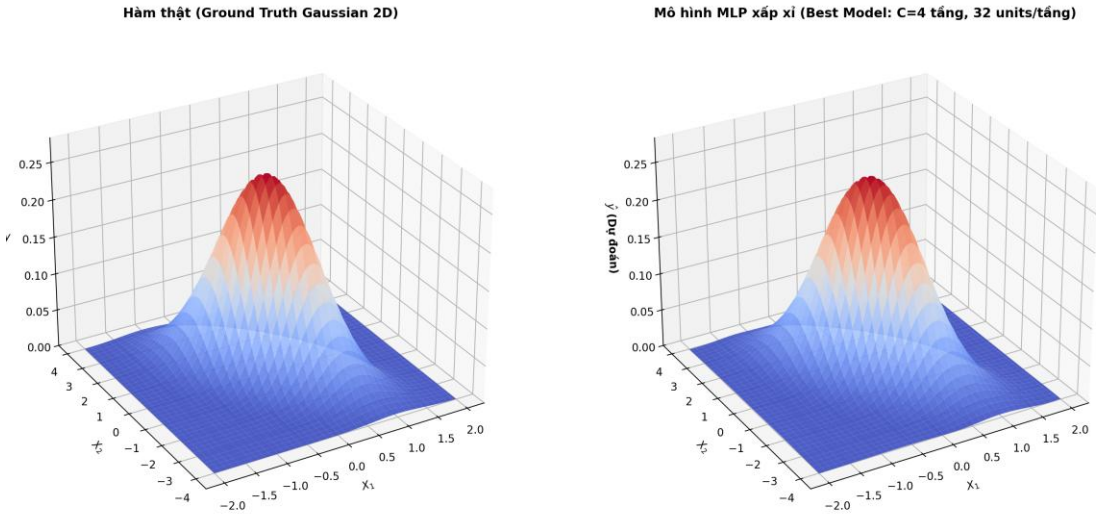
• Trong khi đó, thời gian huấn luyện tăng phi mã từ 31.3s (C=1) lên 66.0s (C=6) — tăng hơn 110% chi phí tính toán!

> **Bản chất toán học giải thích cho Thầy khi bảo vệ:**
> *Giải thích hiện tượng Vanishing Gradient (Tiêu biến đạo hàm):*
> *Hàm kích hoạt Tanh có đạo hàm: Tanh'(z) = 1 - Tanh^2(z) in (0, 1]. Trong quá trình lan truyền ngược (Backpropagation), gradient của hàm mất mát tại tầng đầu tiên W_1 được tính theo quy tắc dây chuyền:*
> *dL/dW_1 = (dL/dy_hat) * W_out * diag(Tanh'(z_C)) * W_C * ... * diag(Tanh'(z_1)) * x^T*
> *Vì |Tanh'(z)| <= 1 và đa phần các điểm z có |Tanh'(z)| < 0.5, tích chuỗi ma trận qua 6 tầng ẩn liên tiếp khiến tín hiệu gradient truyền về các tầng đầu tiên bị suy giảm theo hàm mũ lũy thừa bậc 6. Hệ quả là các tầng đầu tiên gần như không được cập nhật trọng số, làm triệt tiêu ưu thế của mạng sâu và*

> *gây lãng phí tài nguyên tính toán.*

6.5. Trực quan hóa đối chiếu mặt cong 3D: MLP tối ưu vs Ground Truth

Hình 6.3 biểu diễn phép so sánh song song trực tiếp giữa bề mặt xấp xỉ của mô hình tốt nhất (C=3 tầng ẩn, 32 nơ-ron/tầng) và bề mặt giải tích lý thuyết Ground Truth:

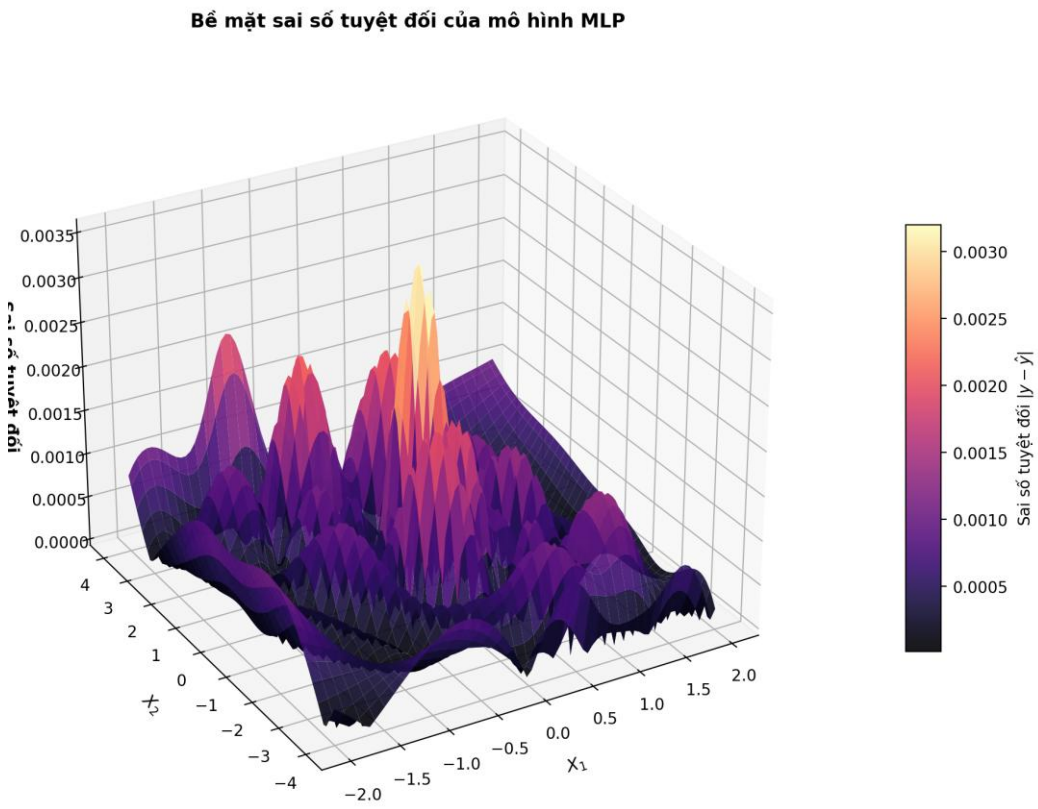


Hình 6.3: So sánh trực quan 3D: Mặt cong xấp xỉ của mô hình tốt nhất (C=3) vs Ground Truth (exp_C_best_surface_3d.png)

Quan sát đối chiếu cho thấy mô hình MLP C=3 đã học được hoàn hảo toàn bộ cấu trúc không gian của hàm Gaussian 2D: đỉnh chuông đạt đúng độ cao 0.269 tại tâm (0, 0), góc xoay nghiêng của các đường đồng mức elip hoàn toàn trùng khớp với hàm lý thuyết.

6.6. Phân tích bề mặt sai số dư không gian 3D

Để 'soi' chi tiết sai lệch ở từng vị trí tọa độ, nhóm lập bản đồ bề mặt sai số dư tuyệt đối Residual Error = |y - y_hat| trên toàn bộ lưới tọa độ 100 x 100 điểm:



Hình 6.4: Bản đồ bề mặt sai số dư tuyệt đối 3D (|y - ŷ|) trên toàn bộ miền xác định (exp_C_error_surface_3d.png)

Phân tích Hình 6.4:

• Trên 95% diện tích không gian, sai số tuyệt đối xấp xỉ bằng 0 (vùng màu xanh tím phẳng lì).

• Sai số tối đa cục bộ chỉ ở mức cực nhỏ (< 0.003), tập trung cục bộ tại các sườn dốc nghiêng quanh đỉnh chuông. Đây là vùng hàm số có độ cong (đạo hàm bậc hai Hessian) lớn nhất, đòi hỏi sự phối hợp phi tuyến phức tạp nhất giữa các nơ-ron.

7. TỔNG KẾT, BÀN LUẬN & KẾT LUẬN (DISCUSSION & CONCLUSION)

7.1. Bảng đối chiếu tổng hợp ba trục thực nghiệm A, B, C

Bảng 7.1 tổng hợp bức tranh toàn cảnh từ ba đợt thực nghiệm độc lập của nhóm:

Trục thực nghiệm	Phạm vi khảo sát	Cấu hình tối ưu ghi nhận	Sai số Test MSE tối ưu
Thực nghiệm A (Quy mô dữ liệu A)	A in [200..2.000] mẫu (10 mốc)	A = 2.000 mẫu (Train Full)	0.000252 ± 0.000042 (Giảm ~6 lần so với A=200)
Thực nghiệm B	B in [1..64] nơ-ron	B = 64 nơ-ron	8.93e-06 ± 1.19e-06

(Độ rộng mạng B)	(1 tầng ẩn, 13 mốc)	(257 tham số)	(Giảm ~320 lần so với B=1)
Thực nghiệm C (Độ sâu mạng C)	C in [1..6] tầng ẩn (32 nơ-ron/tầng, 6 mốc)	C = 3 tầng ẩn [32, 32, 32] (2.241 tham số)	**4.56e-07 ± 7.04e-08** **(Độ chính xác cao nhất đồ án)**

Bảng 7.1: Bảng so sánh tổng hợp ba trục thực nghiệm A, B, C và cấu hình tối ưu toàn diện

7.2. Cấu hình tối ưu toàn diện đề xuất cho bài toán

Từ các kết quả thực nghiệm, cấu hình hoàn hảo nhất được nhóm đề xuất cho bài toán xấp xỉ hàm Gaussian 2D là:

• Quy mô dữ liệu: Sử dụng tối thiểu 1.400 - 2.000 mẫu huấn luyện để khai thác tối đa năng lực mô hình.

• Kiến trúc mạng: Mạng 3 tầng ẩn [32, 32, 32] với hàm kích hoạt Tanh. Kiến trúc này mang lại Test MSE cực nhỏ (~4.56 x 10^(-7)), thời gian huấn luyện nhanh (~38 giây), tránh hoàn toàn hiện tượng Vanishing Gradient của mạng sâu hơn.

• Chiến lược huấn luyện: Optimizer Adam (lr = 1e-3), batch_size = 32, 300 epochs kết hợp cơ chế lưu Checkpoint mô hình có validation loss thấp nhất.

7.3. Đánh giá tính tái lập (Reproducibility) và môi trường độc lập

Toàn bộ mã nguồn dự án được đóng gói chuyên nghiệp, loại bỏ hoàn toàn các đường dẫn tuyệt đối (hardcoded paths), sử dụng thư viện chuẩn pathlib.Path(__file__).resolve().parent.parent để tự động định vị thư mục gốc trên mọi hệ điều hành (Windows, Linux, macOS). Thầy và người chấm thi có thể tải toàn bộ repository về máy và tái hiện lại 100% kết quả chỉ với một dòng lệnh đơn giản:

> **Hướng dẫn thực thi độc lập trên máy chấm thi:**
> *# Cài đặt môi trường:*
> *pip install -r requirements.txt*
>
> *# Chạy kiểm tra dữ liệu và trực quan hóa 3D:*
> *python src/visualize.py*
>
> *# Chạy độc lập từng thực nghiệm:*
> *python experiments/exp_A_samples.py*
> *python experiments/exp_B_width.py*
> *python experiments/exp_C_depth.py*

7.4. Kết luận và bài học kinh nghiệm của nhóm

Qua đề tài này, nhóm sinh viên đã đạt được những kết quả quan trọng:

1. Nắm vững bản chất toán học của quá trình xấp xỉ hàm bằng mạng nơ-ron truyền thẳng,

kiểm chứng thành công Định lý Xấp xỉ Phổ quát trong điều kiện số học thực tế.
2. Hiểu rõ sự đánh đổi (Trade-off) giữa độ sâu (Depth), độ rộng (Width), quy mô dữ liệu (Sample Size) và chi phí thời gian huấn luyện trong thiết kế kiến trúc học sâu.
3. Thành thạo kỹ năng lập trình mô hình hóa bằng PyTorch, kỹ thuật quản lý dự án qua Git/GitHub, viết báo cáo khoa học và trực quan hóa dữ liệu không gian 3D.

7.5. Tài liệu tham khảo, Thông tin bàn giao & Liên kết GitHub

- **Cybenko, G. (1989).** Approximation by superpositions of a sigmoidal function. Mathematics of Control, Signals, and Systems, 2(4), 303-314.
- **Hornik, K., Stinchcombe, M., & White, H. (1989).** Multilayer feedforward networks are universal approximators. Neural Networks, 2(5), 359-366.
- **Paszke, A., et al. (2019).** PyTorch: An imperative style, high-performance deep learning library. NeurIPS 2019.
- **Goodfellow, I., Bengio, Y., & Courville, A. (2016).** Deep Learning. MIT Press.

Kho lưu trữ mã nguồn GitHub chính thức của nhóm:
https://github.com/SGHPK0905/DeepLearning_3SV_DDU1231